

05506002 Methods of Numerical Computation

Lab 1

Objectives

1. Start writing Python using Google Colab
2. Observe round-off errors and truncation errors

Google Colab

Google Colab, short for Google Colaboratory, is a free cloud service by Google that allows users to write and execute Python or R code through your web browser. It is particularly useful for data analysis, machine learning, and computational tasks, providing access to powerful computing resources such as GPUs and TPUs without the need for complex setup or installation. Users can easily share their notebooks with others, making it a collaborative tool that enhances productivity and learning.

In this subject, we will use Python in the first half of the semester, then we will switch to Julia which is another programming language which is mostly used for numerical analysis and computational science.

You don't need to worry about Julia for now...

For those who have not tried Google Colab, you can navigate to <https://colab.google/> and click on "New Notebook" button.

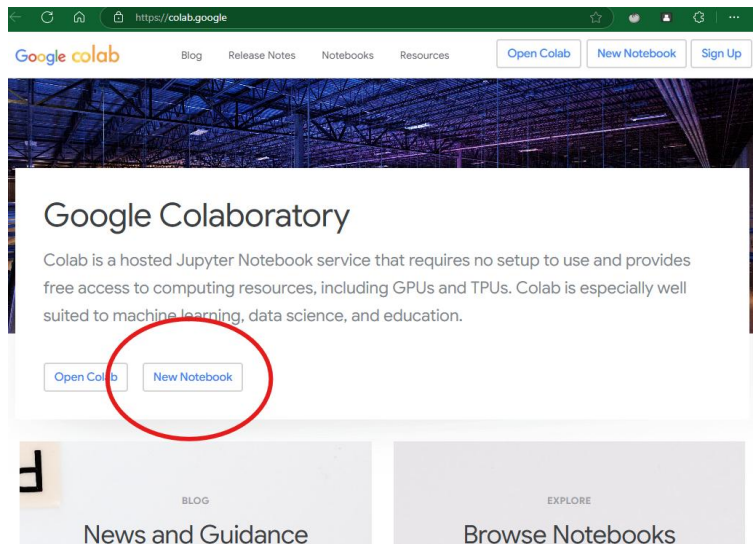


Figure 1 – Google Colab starting screen

Sign in with your KMITL email address and you will be ready to go.

Recapping Python

For those who are not familiar with Python, the following codes will make you recap the language in no time.

The following code is how you use for loop to print the values of x^2 .

```
# Compute squares using a loop
for i in range(1, 11):
    print(f"{i}^2 = {i*2}")
1^2 = 1
2^2 = 4
3^2 = 9
4^2 = 16
5^2 = 25
6^2 = 36
7^2 = 49
8^2 = 64
9^2 = 81
10^2 = 100
```

When you need to write a function, here is a function to calculate factorial of n.

```
# Define a function for factorial
def factorial(n):
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result

# Test the factorial function
print("Factorial of 5:", factorial(5))
Factorial of 5:120
```

It is effective to use numpy as array.

```
import numpy as np

# Create an array
arr = np.arange(1, 11)

# Compute the square of each element
squared = arr**2
print("Squares:", squared)

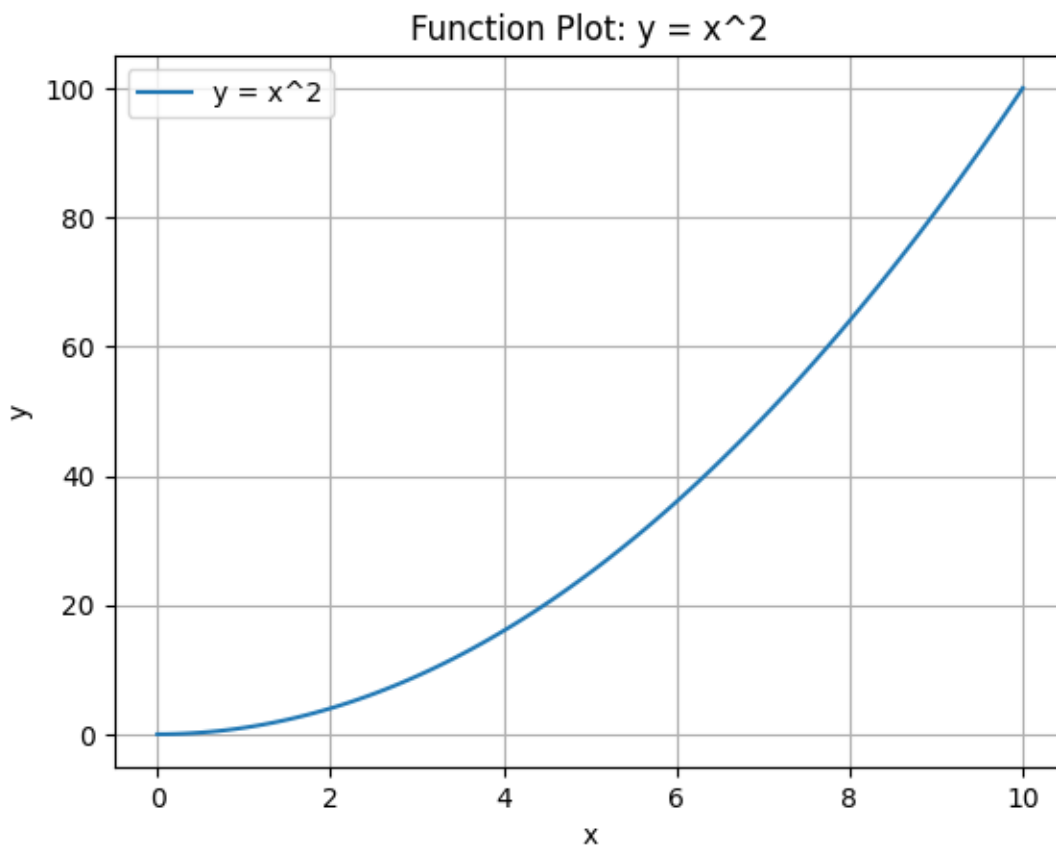
# Compute the sum of elements
total_sum = np.sum(arr)
print("Sum of array elements:", total_sum)
Squares: [ 1  4  9 16 25 36 49 64 81 100]
Sum of array elements: 55
```

When you need to plot a graph, you can use PyPlot in Matplotlib.

```
import matplotlib.pyplot as plt

# Define the function
x = np.linspace(0, 10, 100)
y = x**2

# Plot the function
plt.plot(x, y, label="y = x^2")
plt.title("Function Plot: y = x^2")
plt.xlabel("x")
plt.ylabel("y")
plt.grid()
plt.legend()
plt.show()
```



Numerical Errors

Round-off and truncation errors are common issues in programming and can significantly affect the accuracy of computations.

Round-off errors occur because computers represent numbers with a finite number of digits. When a number exceeds the available precision, it is rounded to the nearest representable value. This can lead to small inaccuracies that accumulate over multiple calculations, especially in floating-point arithmetic.

We can demonstrate the round-off errors as follows – **CASE 1**

```
x = 1 / 49
print(x)
0.02040816326530612
```

```
x = x * 49
print(x)
0.9999999999999999
```

Another case is as follows – **CASE 2**

```
a = 0.1
b = a + a + a

print(b == 0.3)
False
```

We can demonstrate the effect of round-off errors by adding a small amount of number (0.00000001) to a large number (10000000). Since floating point can choose only large number over the fraction, adding these numbers together will cause round-off errors. When we add this small number more and more, the errors will keep increasing as follows.

```
import numpy as np
import matplotlib.pyplot as plt

# Parameters
large_number = 1e8
small_increment = 1e-8
n_steps = 1000

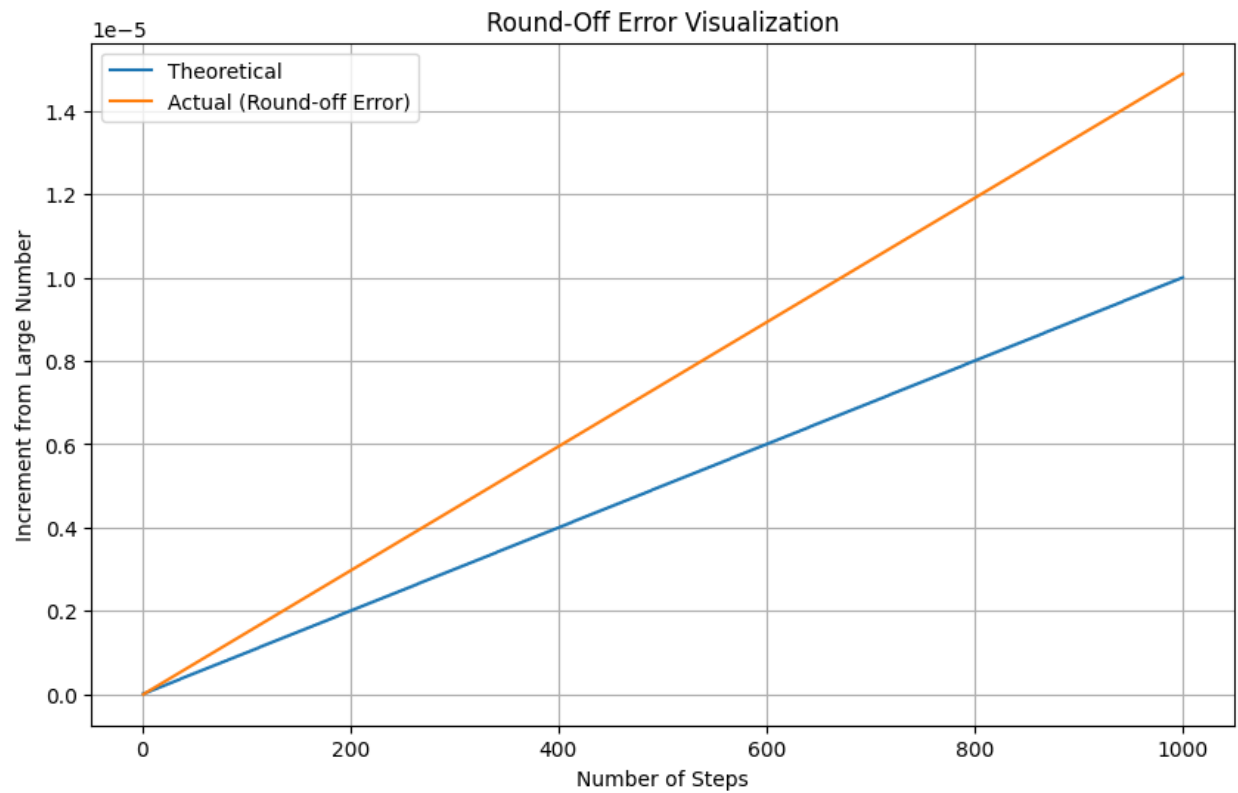
# Compute results
theoretical = large_number + np.arange(1, n_steps + 1) * small_increment
actual = np.full(n_steps, large_number)
```

```

for i in range(1, n_steps):
    actual[i] = actual[i - 1] + small_increment

# Plot results
plt.figure(figsize=(10, 6))
plt.plot(range(1, n_steps + 1), theoretical - large_number, label="Theoretical")
plt.plot(range(1, n_steps + 1), actual - large_number, label="Actual (Round-off Error)")
plt.title("Round-Off Error Visualization")
plt.xlabel("Number of Steps")
plt.ylabel("Increment from Large Number")
plt.legend()
plt.grid()
plt.show()

```



Truncation errors, on the other hand, arise when an infinite or long decimal number is approximated by cutting off digits beyond a certain point. This is common in iterative methods and series approximations, where an infinite process is approximated by a finite number of steps.

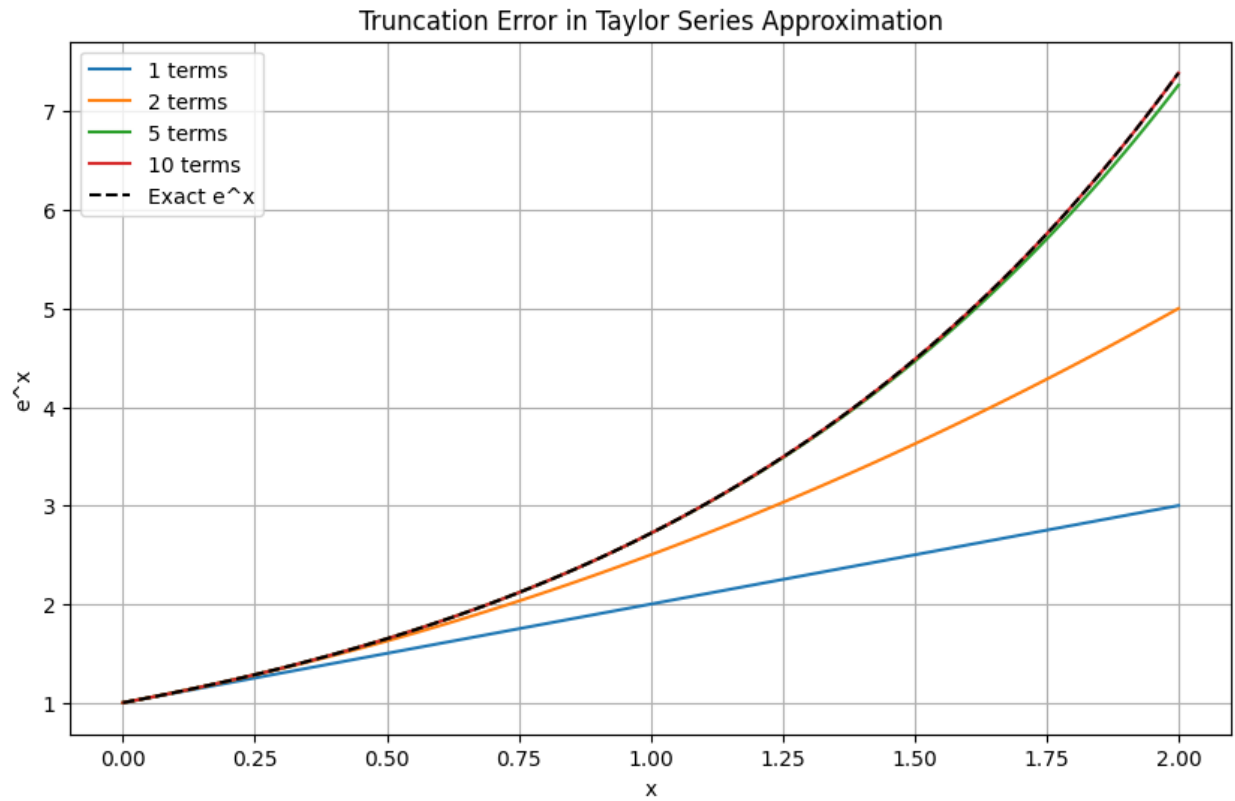
The following example is Taylor series of e^x . A Taylor series with fewer terms tends to have more errors. – **CASE 3**

```
# Function to compute Taylor series approximation
def taylor_exp(x, n_terms):
    approximation = np.zeros_like(x)
    for n in range(n_terms + 1):
        approximation += x**n / factorial(n)
    return approximation

# Parameters
x = np.linspace(0, 2, 100) # x values to approximate e^x
exact = np.exp(x) # Exact values
n_terms_list = [1, 2, 5, 10] # Different numbers of terms

# Plot results
plt.figure(figsize=(10, 6))
for n_terms in n_terms_list:
    approx = taylor_exp(x, n_terms)
    plt.plot(x, approx, label=f"{n_terms} terms")

plt.plot(x, exact, "k--", label="Exact e^x")
plt.title("Truncation Error in Taylor Series Approximation")
plt.xlabel("x")
plt.ylabel("e^x")
plt.legend()
plt.grid()
plt.show()
```

**Tasks (Submit through Google Classroom)**

1. Calculate the true error and true relative error of CASE 1, 2, and 3 above using Python and print the result.